Name: Jai Navani
Class: D20A
Roll No: 36
BlockChain & DLT

EXPERIMENT NO - 6

AIM :- Creating Smart Contracts and performing transactions using Solidity and Remix IDE

THEORY :-

1. What is a Smart Contract?

Answer:
A smart contract is a self-executing program stored on the blockchain that automatically executes when predefined conditions are met. It ensures secure and transparent transactions without intermediaries.

2. How was the Smart Contract implemented in this experiment?

Answer:
In this experiment, a smart contract was created using Solidity in Remix IDE. It stores student details such as name, course, and marks, and allows retrieval of this data using functions.

3. What is the purpose of the addStudent function?

Answer:
The addStudent function is used to store student data on the blockchain. It takes inputs like student ID, name, course, marks, and stores them securely in the contract.

4. What is the purpose of the getStudent function?

Answer:
The getStudent function is used to retrieve stored student data from the blockchain. It ensures that only existing records are accessed and returns the stored details.

5. What is the role of IPFS in this experiment?

Answer:
IPFS is used to store certificate images or documents. Instead of storing large files on the blockchain, only the IPFS hash is stored in the smart contract, making the system efficient and secure.

Implementation & Output :-

Step 1: Creation of Smart Contracts

Smart contracts named **AcademicRecords.sol** and **Verification.sol** were created using Solidity in Remix IDE. These contracts are used to store student academic records and verify certificate authenticity on the blockchain.

AcademicRecords.sol

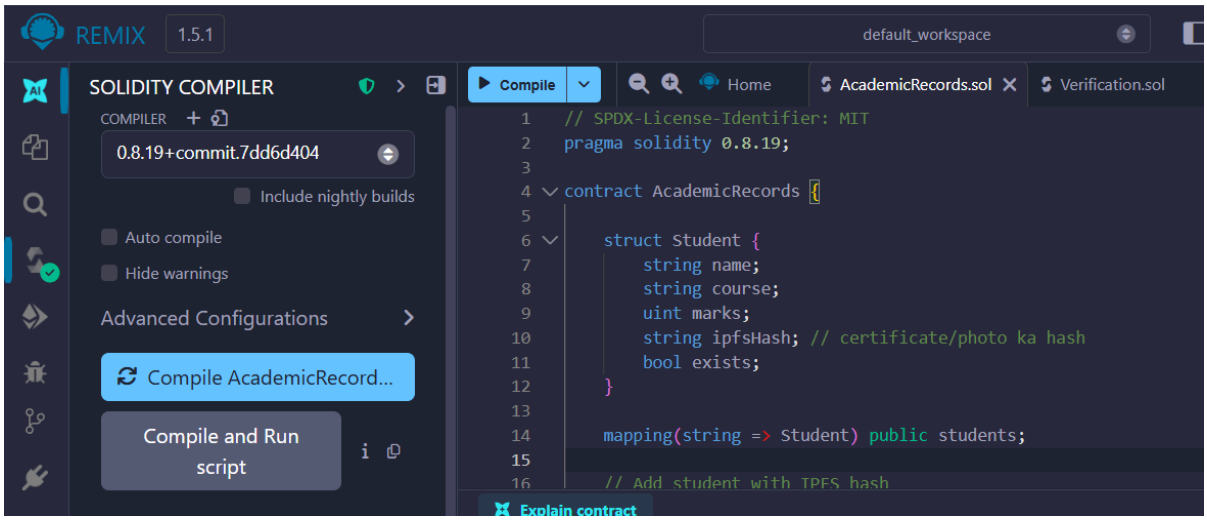
```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.19;
contract AcademicRecords {

  struct Student {
      string name;
      string course;
      uint marks;
      string ipfsHash;
      bool exists;
  }

  mapping(string => Student) public students;

  function addStudent(
      string memory _id,
      string memory _name,
      string memory _course,
      uint _marks,
      string memory _ipfsHash
  ) public {
      students[_id] = Student(_name, _course, _marks, _ipfsHash, true);
  }

  function getStudent(string memory _id)
      public view
      returns (string memory, string memory, uint, string memory)
  {
      require(students[_id].exists, "Student not found");
      Student memory s = students[_id];
      return (s.name, s.course, s.marks, s.ipfsHash);
  }
}
```

Verification.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract Verification {

  mapping(string => bool) public verified;

  function verifyCertificate(string memory _id) public {
     verified[_id] = true;
  }

  function checkVerification(string memory _id)
     public view
     returns (bool)
  {
     return verified[_id];
  }
}
```

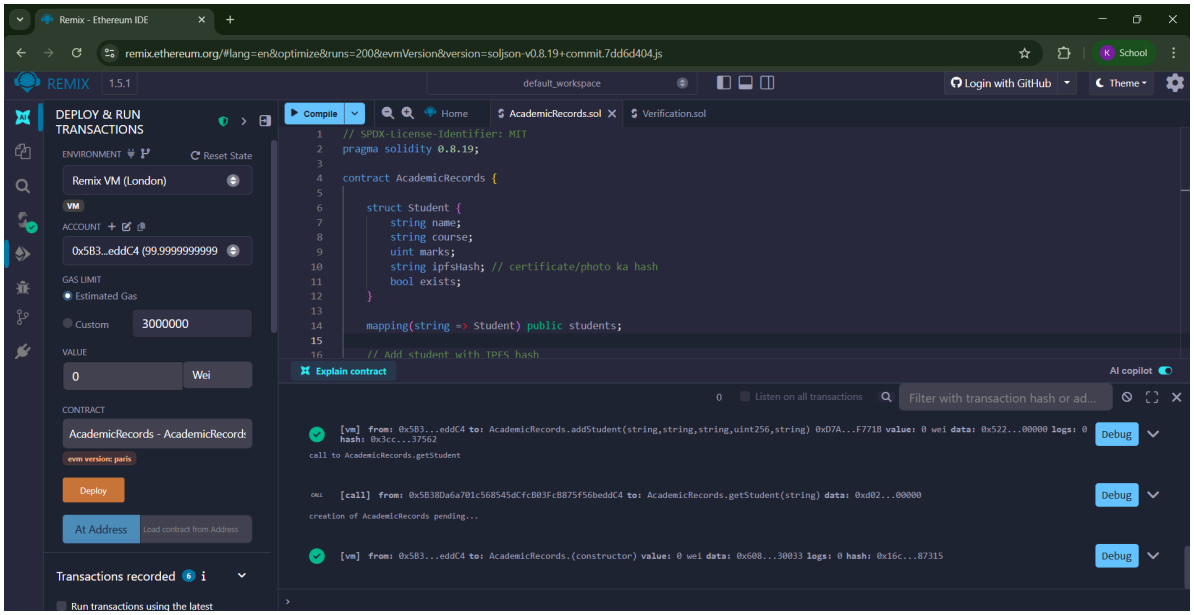
Step 2: Compilation of Smart Contracts

Both smart contracts were compiled using the Solidity Compiler in Remix IDE. The compiler version 0.8.19 was selected to ensure compatibility and successful compilation.



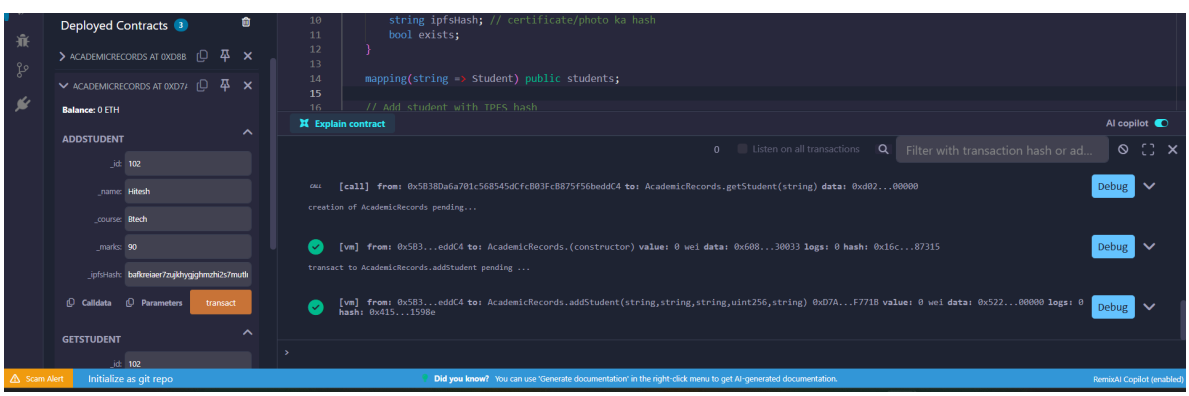
Step 3: Deployment of Smart Contracts

The contracts were deployed using the “Deploy & Run Transactions” module in Remix IDE with the JavaScript VM environment. Successful deployment generated a contract address for interaction.



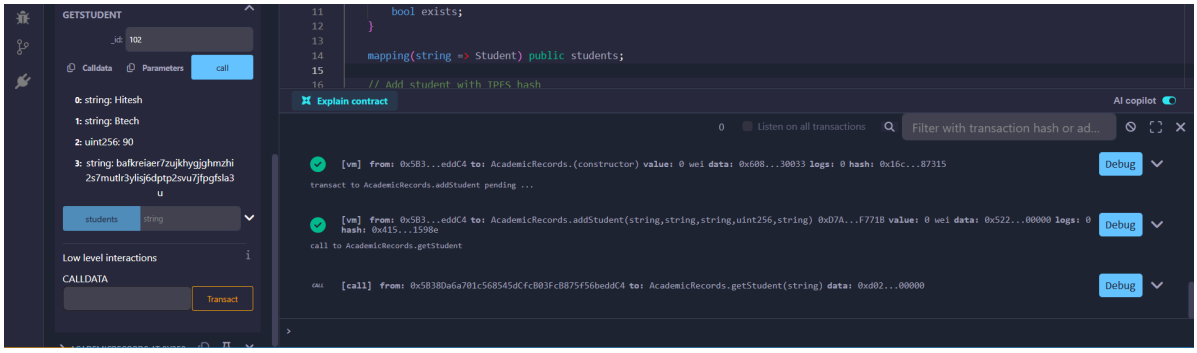
Step 4: Execution of AcademicRecords Contract

The addStudent function was executed to store student details including ID, name, course, marks, and IPFS hash. The getStudent function was then used to retrieve and verify the stored data from the blockchain.



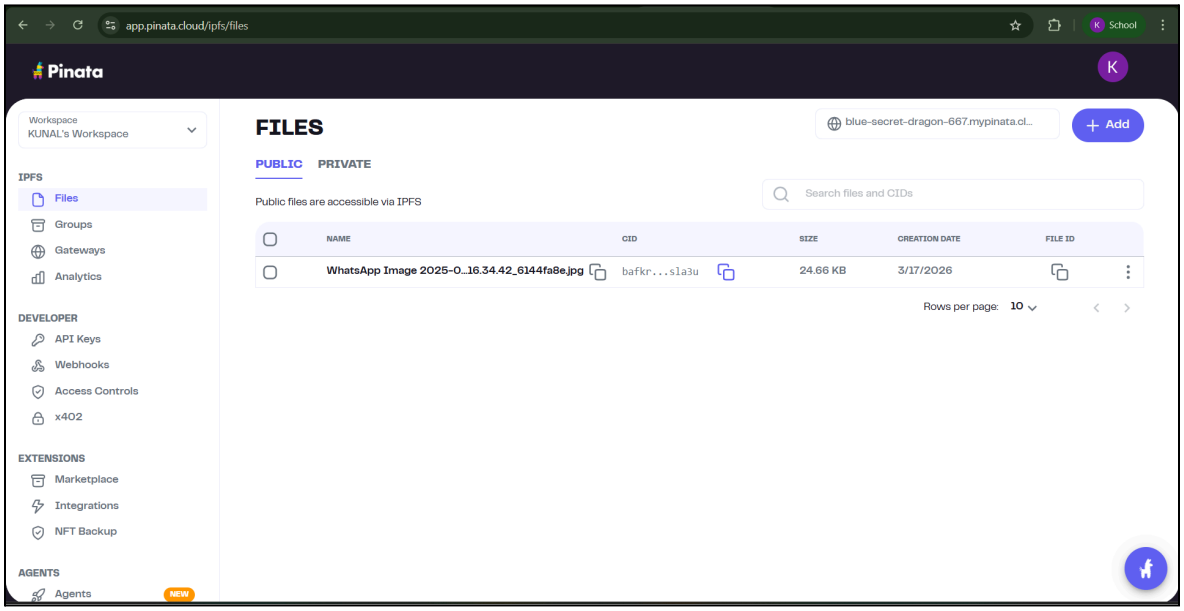
Step 5: Execution of Verification Contract

The verifyCertificate function was used to mark a certificate as verified. The checkVerification function was used to confirm whether a particular certificate is verified or not.



Step 6: Integration with IPFS

Student certificate images were uploaded to IPFS using a pinning service. The generated IPFS hash was stored in the smart contract, enabling secure storage and retrieval of documents without storing large files directly on the blockchain.



Conclusion :-

This experiment helped in understanding the practical implementation of smart contracts using Solidity in Remix IDE. A smart contract was successfully designed to store and retrieve student academic records on the blockchain, ensuring data security and immutability. The system was further enhanced by integrating IPFS to store certificate images, with only the hash stored on the blockchain. This approach makes the system efficient, secure, and resistant to data tampering. Overall, the experiment demonstrated how blockchain technology can be effectively used for reliable academic credential verification.